

9

WRITING HAIKU WITH MARKOV CHAIN
ANALYSIS

Computers can write poetry by rearranging existing poems. This is basically what humans do. You and I didn't invent the language we speak—we learned it. To talk or write, we just recombine existing words—and rarely in a truly original manner. As Sting once said about writing music, “I don't think there's such a thing as composition in pop music. I think what we do in pop music is collate . . . I'm a good collator.”

In this chapter, you're going to write a program that puts the “best words in the best order” in the form of haiku. But to do this, Python needs good examples, so you'll need to provide a training corpus of haiku by the Japanese masters.

To rearrange these words in a meaningful manner, you will use *Markov chains*, named after Russian mathematician Andrey Markov. *Markov chain analysis*, an important part of probability theory, is a process that attempts to predict the subsequent state based on the properties of the current state. Modern-day applications include speech and handwriting recognition, computer performance evaluation, spam filtering, and Google's PageRank algorithm for searching the web.

With Markov chain analysis, a training corpus, and the syllable-counting program from **Chapter 8**, you'll be able to produce new haiku that follow the syllabic rules of the genre and stay "on subject" to a large degree. You'll also learn how to use Python's `logging` module to help monitor the behavior of your program with easy on-and-off feedback. And in "**Challenge Projects**" on **page 184**, you can enlist your friends on social media to see if they can distinguish your simulated haiku from the real thing.

Project #16: Markov Chain Analysis

Like the genetic algorithms in **Chapter 7**, Markov chain analysis sounds impressive but is easy to implement. You do it every day. If you hear someone say, "Elementary, my dear . . .," you automatically think, "Watson." Every time your brain has heard this phrase, it has taken a sample. Based on the number of samples, it can predict the answer. On the other hand, if you heard someone say, "I want to go to . . .," you might think "the bathroom" or "the movies" but probably not "Houma, Louisiana." There are many possible solutions, but some are more likely than others.

Back in the 1940s, Claude Shannon pioneered the use of Markov chains to statistically model the sequences of letters in a body of text. For example, for every occurrence of the digram *th* in an English-language book, the next most likely letter is *e*.

But you don't just want to know what the most likely letter is; you want to know the actual probability of getting that letter, as well as the odds of getting every other letter, which is a problem tailor-made for a computer. To solve this problem, you need to map each two-letter digram in a piece of text to the letter that immediately follows it. This is a classic dictionary application, with the digrams as the keys and the letters as the values.

When applied to letters in words, a *Markov model* is a mathematical model that calculates a letter's probability of occurrence based on the previous *k* consecutive letters, where *k* is an integer. A *model of order 2* means that the probability of a letter occurring depends on the two letters that precede it. A *model of order 0* means that each letter is indepen-

dent. And this same logic applies to words. Consider these two haiku examples:

A break in the clouds

Glorious the moon

The moon a bright
mountaintop

Therefore our thanks dark clouds
come

Distant and aloof

To rest our tired necks

A Python dictionary that maps each haiku word to each subsequent word looks like this:

```
'a': ['break', 'bright'],  
'aloof': ['glorious'],  
'and': ['aloof'],  
'break': ['in'],  
'bright': ['mountaintop'],  
'clouds': ['the', 'come'],  
'come': ['to'],  
'dark': ['clouds'],  
'distant': ['and'],  
'glorious': ['the'],  
'in': ['the'],  
'moon': ['a', 'therefore'],  
'mountaintop': ['distant'],  
'our': ['thanks', 'tired'],  
'rest': ['our'],  
'thanks': ['dark'],  
'the': ['clouds', 'moon', 'moon'],  
'therefore': ['our'],  
'tired': ['necks'],  
'to': ['rest']
```

Since there are only two haiku, most of the dictionary keys have only one value. But look at *the* near the bottom of the list: *moon* occurs twice. This

is because the Markov model stores every occurrence of a word as a separate, duplicate value. So, for the key *the*, if you choose a value at random, the odds of selecting *moon* versus *clouds* are 2:1. Conversely, the model will automatically screen out extremely rare or impossible combinations. For example, many words can potentially follow *the*, but not another *the*!

The following dictionary maps every *pair* of words to the word immediately after; that means it's a model of order 2.

```
'a break': ['in'],
'a bright': ['mountaintop'],
'aloof glorious': ['the'],
'and aloof': ['glorious'],
'break in': ['the'],
'bright mountaintop': ['distant'],
'clouds come': ['to'],
'clouds the': ['moon'],
'come to': ['rest'],
'dark clouds': ['come'],
'distant and': ['aloof'],
'glorious the': ['moon'],
'in the': ['clouds'],
'moon a': ['bright'],
'moon therefore': ['our'],
'mountaintop distant': ['and'],
'our thanks': ['dark'],
'our tired': ['necks'],
'rest our': ['tired'],
'thanks dark': ['clouds'],
'the clouds': ['the'],
'the moon': ['a', 'therefore'],
'therefore our': ['thanks'],
'to rest': ['our']
```

Note that the mapping continues from the first haiku to the second so the dictionary contains the items 'and aloof': ['glorious'] and 'aloof glorious': ['the']. This behavior means your program can jump from one haiku to

another and is not restricted to just the word pairs within a single haiku. It is free to form new word pairs that the masters may never have conceived.

Because of the very short training corpus, *the moon* is the only word pair with multiple values. For all the others, you are “locked in” to a single outcome. In this example, the size of the training corpus greatly determines the number of values per key, but with a larger corpus, the value of k in the Markov model will have a larger influence.

The size of k determines whether you produce poppycock, commit plagiarism, or produce a perspicuous piece of originality. If k equals 0, then you’ll be choosing words at random based on that word’s overall frequency in the corpus, and you’ll likely produce a lot of gibberish. If k is large, the results will be tightly constrained, and you’ll begin to reproduce the training text verbatim. So small values of k promote creativity, and large values promote duplication. The challenge is finding the proper balance between the two.

To illustrate, if you use a Markov model of order 3 on the previous haiku, all the resulting keys will have one value. The two values associated with the word pair *the moon* are lost because the former word pair becomes two keys, each with a unique value:

```
'the moon a': ['bright'],  
'the moon therefore': ['our']
```

Since haiku are short—only 17 syllables long—and available training corpora are relatively small, using a k of 2 should be sufficient to enforce *some* order while still allowing for creative word substitutions in your program.

THE OBJECTIVE

Write a program that generates haiku using Markov chain analysis. Allow the user to modify the haiku by independently regenerating lines two and three.

The Strategy

Your general strategy for simulating haiku will be to build Markov models of orders 1 and 2 with a training corpus of haiku written by humans. You'll then use those models and the *count_syllables.py* program from **Chapter 8** to generate novel haiku that meet the required syllabic structure of 5-7-5 for the three lines of the haiku.

The program should build the haiku one word at a time, initiating (or *seeding*) the haiku with a random word drawn from the corpus; selecting the haiku's second word using a Markov model of order 1; and then selecting each subsequent word with the order 2 model.

Each word is derived from a *prefix*—a word or word pair that determines which word will be picked to go in the haiku; the key in the word-mapping dictionaries represents the prefix. As a consequence, the word that the prefix determines is the *suffix*.

Choosing and Discarding Words

When the program selects a word, it first counts the word's syllables, and if the word doesn't fit, it chooses a new word. If there are no possible words based on the prefix in the poem, then the program resorts to what I call a *ghost prefix*, which is a prefix that doesn't occur in the haiku. For example, if a word pair in a haiku is *temple gong*, and all the words that follow in the Markov model have too many syllables to complete the line, the program selects a new word pair at random and uses it to pick the next word in the haiku. The new word-pair prefix *should not be included in the line*—that is, *temple gong* will not be replaced. Although you could choose a suitable new word in a number of ways, I prefer this technique because it allows you to simplify by maintaining a consistent process throughout the program.

You can accomplish these steps with the functions in **Figures 9-1** and **9-2**. Assuming you're working on a five-syllable line, **Figure 9-1** is an example of what will happen, at a high level, if all the chosen words match the syllable target.

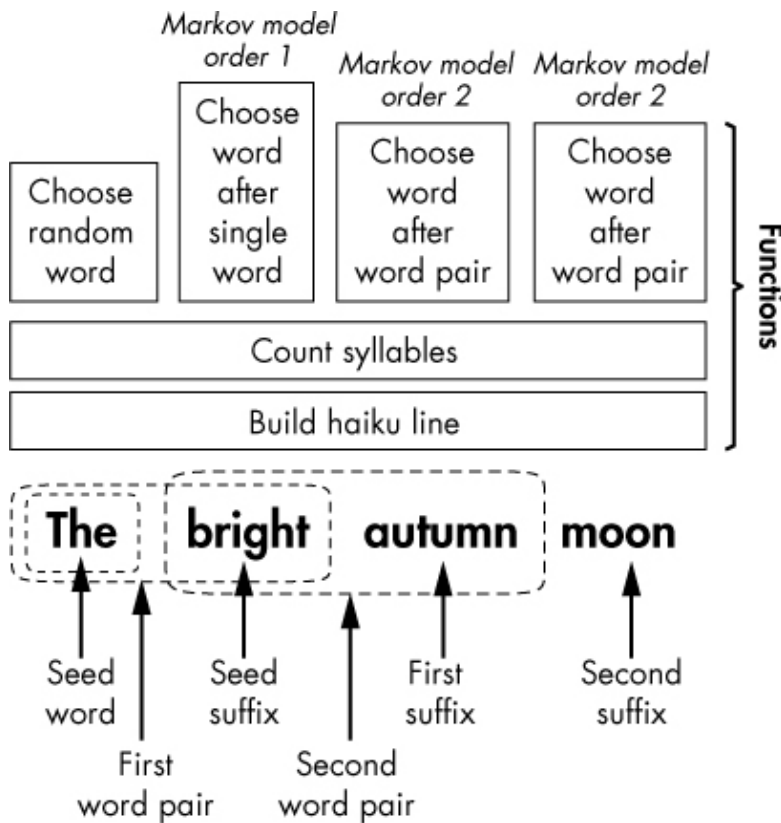


Figure 9-1: High-level graphical pseudocode for a five-syllable haiku line

The program randomly selects the seed word *the* from the corpus, and then counts its syllables. Next, it chooses *bright* from the model of order 1, based on the prefix *the*. Then it counts the number of syllables in *bright* and adds that number to the number of syllables in the line. Since the sum of syllables doesn't exceed five, the program adds *bright* to the line, moves on to select *autumn* from the model of order 2 based on the prefix *The bright*, and then repeats the syllable-counting process. Finally, the program selects *moon* based on the prefix *bright autumn*, counts the syllables, and—since the line's total number of syllables is equal to five—adds *moon* to the line, completing it.

Figure 9-2 demonstrates a case where the program needs to utilize a ghost prefix to successfully complete a five-syllable line.

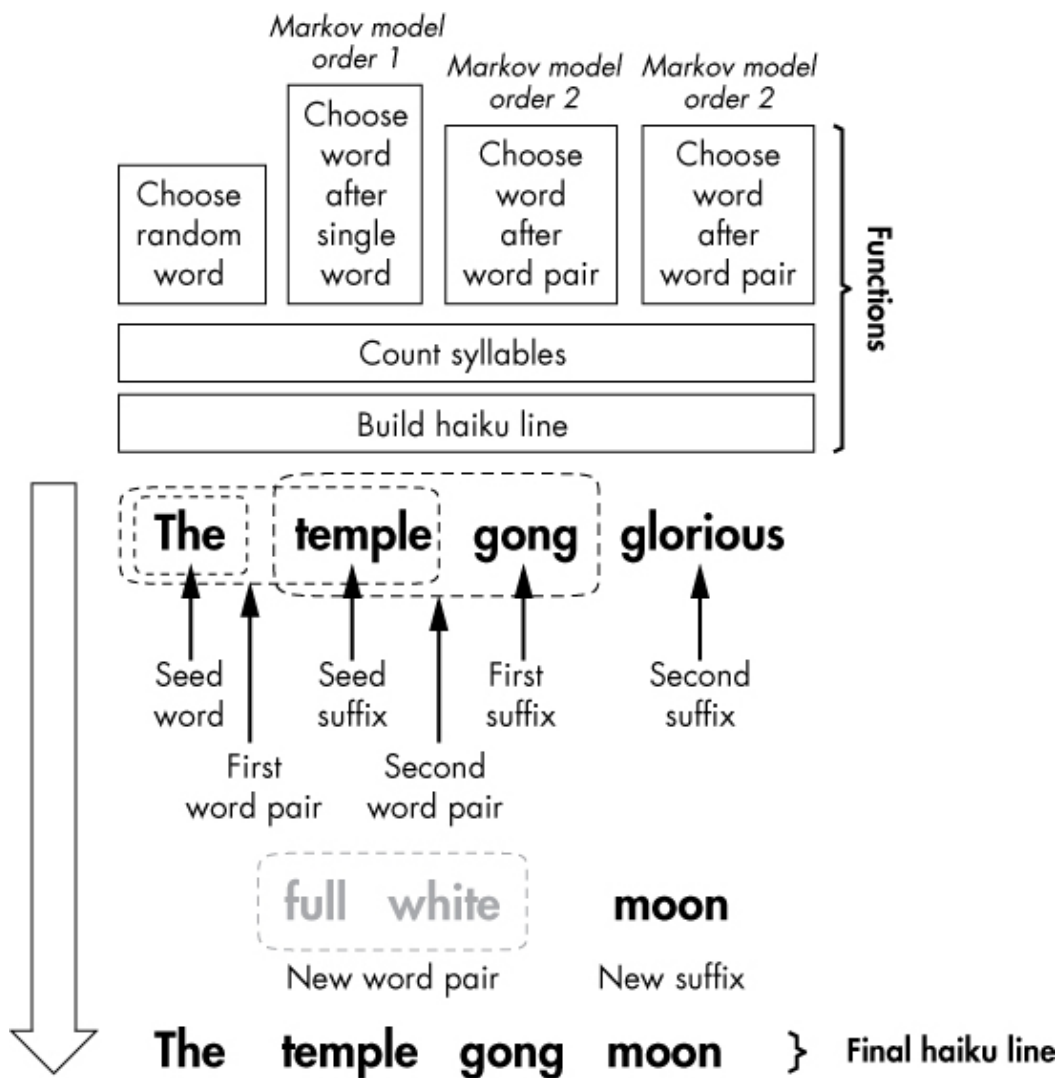


Figure 9-2: Choosing a new suffix with a randomly selected ghost prefix (full white)

Let's assume that the only word that follows the prefix *temple gong* in the Markov model is *glorious*. This word has too many syllables for the line, so the program selects a ghost prefix, *full white*, at random. The word *moon* follows the ghost prefix and satisfies the remaining syllable count in the line, so the program adds it to the line. The program then discards the *full white* prefix, and the line is complete. With this ghost prefix technique, you can't guarantee that the new suffix will make sense contextually, but at the same time, this is one way to incorporate creativity into the process.

Continuing from One Line to Another

The Markov model is the "special sauce" that allows you to imbue the haiku with context and meaning that continue from one line to another. The Japanese masters *generally* wrote haiku in which each line is a stand-

alone phrase but the contextual thread continues across lines, as in this haiku from Bon Cho:

In silent midnight

Our old scarecrow topples down

Weird hollow echo

—*Bon Cho*

Even though the masters preferred that each line of a haiku represent a complete thought, they didn't strictly follow the rule. Here's an example in Buson's haiku:

My two plum trees are

So gracious see, they flower

One now, one later

—*Buson*

The first line of Buson's haiku is not grammatical on its own, so the reader must continue to the next line without a break. When a phrase in poetry moves from one line to the next without a pause or syntactic break, it is said to be *enjambéd*. According to Charles Hartman, author of *Virtual Muse*, enjambment is what gives metrical lines much of their supple liveliness. That's a good thing, since it's very hard to get an algorithm to write a coherent poem without some grammatical spillover from line to line. To get your program to continue a "thought" through multiple lines, you need to use the word pair from the end of the previous line as the starting prefix for the current line.

Finally, you should give the user the opportunity to not only build the poem but also to edit it interactively by regenerating lines two and three. Most of writing is rewriting, and it would be unconscionable to leave the user hanging with two perfect lines and no way to throw the dice again on an uncooperative line.

The Pseudocode

If you follow the strategy I've just laid out, your high-level pseudocode should look like this:

```
Import count_syllables module
Load a training-corpus text file
Process the training corpus for spaces, newline breaks, and so on
Map each word in corpus to the word after (Markov model order 1)
Map each word pair in corpus to the word after (Markov model order 2)
Give user choice of generating full haiku, redoing lines 2 or 3, or exiting
If first line:
    Target syllables = 5
    Get random word from corpus <= 4 syllables (no 1-word lines)
    Add word to line
    Set random word = prefix variable
    Get mapped words after prefix
    If mapped words have too many syllables
        Choose new prefix word at random & repeat
    Choose new word at random from mapped words
    Add the new word to the line
    Count syllables in word and calculate total in line
    If syllables in line equal target syllables
        Return line and last word pair in line
Else if second or third line:
    Target = 7 or 5
    Line equals last word pair in previous line
    While syllable target not reached:
        Prefix = last word pair in line
        Get mapped words after word-pair prefix
        If mapped words have too many syllables
            Choose new word-pair prefix at random and repeat
        Choose new word at random from mapped words
        Add the new word to the line
        Count syllables in word and calculate total in line
        If total is greater than target
            Discard word, reset total, and repeat
```

```
    Else if total is less than target
        Add word to line, keep total, and repeat
    Else if total is equal to target
        Add word to line
    Return line and last word pair in line
Display results and choice menu
```

The Training Corpus

Markov models are built from a corpus, so they are unique to that corpus. A model built from the complete works of Edgar Rice Burroughs will be different and distinguishable from one built from the works of Anne Rice. We all have a signature style, or *voice*, and given a large enough sample, a Markov approach can produce a statistical model of your style. Like fingerprints, this model can link you to a document or manuscript.

To build the Markov models, the corpus you'll use is a text file consisting of almost 300 ancient and modern haiku, more than 200 of which were written by the masters. Ideally, your training corpus would consist of thousands of haiku, all by the same author (for a consistent voice), but these are difficult to find, particularly since many of the old Japanese haiku don't obey the syllabic rules, either intentionally or as a result of translation into English.

To increase the number of values per key in the Markov model, the haiku in the initial corpus were duplicated 18 times and randomly distributed throughout the file. This has no impact on word associations *within* haiku but increases the interactions *between* haiku.

To illustrate, assume the word pair at the end of the following haiku is unique, mapping only to the starting word of the second haiku; this results in the fairly useless key/value pair of 'hollow frog': ['mirror-pond']:

Open mouth reveals

Your whole wet interior

Silly **hollow frog!**

Mirror-pond of stars

Suddenly a summer shower

Dimples the water

If you duplicate and shuffle the haiku, you may introduce a preposition into the mix, greatly increasing the odds of linking the odd *hollow frog* to something sensible:

Open mouth reveals

Your whole wet interior

Silly **hollow frog**!

In the city fields

Contemplating cherry trees

Strangers are like friends

The Markov model now assigns two values to 'hollow frog': 'mirror-pond' and 'in'. And each time you duplicate the haiku, you'll see an increase in the number of values per key for haiku-ending words or word pairs. But this is helpful only to a point; after a while, diminishing returns set in, and you start adding the same values over and over again, gaining nothing.

Debugging

Debugging is the process of finding and fixing errors (bugs) in computer hardware and software. When you're trying to code a solution to a complex problem, you need to keep a tight rein on your program in order to find the source of an issue when something unexpected arises. For example, if you end up with seven syllables rather than five in the first line of your haiku, you'll want to know if the syllable-counting function failed, if there was a problem mapping words to words, or if the program thought it was on line two. To find out what went wrong, you need to monitor what your program is returning at each key step, and this calls for either

scaffolding or *logging*. I'll discuss each of these techniques in the following two sections.

Building the Scaffolding

Scaffolding, as defined here, is temporary code you write to help develop your programs, then delete when you're done. The name is an allusion to the scaffolding used in construction—necessary, but nobody wants it around forever.

One common piece of scaffolding is a `print()` statement that checks what a function or calculation returns. The user doesn't need to see the output, so you delete it after you confirm the program is working properly.

Useful scaffolding output includes things like the type of a value or variable, the length of a dataset, and the results of incremental calculations. To quote Allen Downey in *Think Python*, “Time you spend building scaffolding can reduce the time you spend debugging.”

The downside to using `print()` statements for debugging is that you have to go back and delete (or comment out) all these statements later, and you risk accidentally removing a `print()` statement that's useful to the end user. Fortunately, there's an alternative to scaffolding that lets you avoid these problems. It's called the `logging` module.

Using the logging Module

The `logging` module is part of the Python Standard Library (<https://docs.python.org/3/library/logging.html>). With `logging`, you can get a customized report on what your program is doing at any location you choose. You can even write the reports to a permanent logfile. The following interactive shell example uses `logging` to check that a vowel-counting program is working correctly:

```
❶ >>> import logging
❷ >>> logging.basicConfig(level=logging.DEBUG,
                           format='%(levelname)s - %(message)s')

>>> word = 'scarecrow'
```

```

>>> VOWELS = 'aeiouy'
>>> num_vowels = 0
>>> for letter in word:
    if letter in VOWELS:
        num_vowels += 1
    ❸ logging.debug('letter & count = %s-%s', letter, num_vowels)

DEBUG - letter & count = s-0
DEBUG - letter & count = c-0
DEBUG - letter & count = a-1
DEBUG - letter & count = r-1
DEBUG - letter & count = e-2
DEBUG - letter & count = c-2
DEBUG - letter & count = r-2
DEBUG - letter & count = o-3
DEBUG - letter & count = w-3

```

To use the `logging` module, first import it ❶. Then set up what debugging information you want to see and in what format ❷. The `DEBUG` level is the lowest level of information and is used for diagnosing the details. Note that the output uses string formatting with `%s`. You can include more information—for example, the date and time are shown using `format='% (asctime)s'`—but for this snippet of code, all you really need to check is whether the program is counting vowels correctly.

For each letter evaluated, enter the custom text message to display along with the variable values. Note that you have to convert nonstring objects, such as integers and lists, to strings ❸. The `logging` output follows. You can see the cumulative count, along with which letters actually change the count.

Like scaffolding, `logging` is for the developer, not the user. And like the `print()` function, `logging` can slow down your program. To disable the `logging` messages, simply insert the `logging.disable(logging.CRITICAL)` call after you import the module, as follows:

```
>>> import logging
>>> logging.disable(logging.CRITICAL)
```

Placing the disable call near the top of the program will allow you to find it easily and toggle messages on and off. The `logging.disable()` function will suppress all messages at the designated level or lower. Since `CRITICAL` is the highest level, passing it to the `logging.disable()` function turns all messages off. This is a far better solution than manually finding and commenting out calls to `print()`!

The Code

The *markov_haiku.py* code in this section will take a training corpus called *train.txt*, prepare Markov models as dictionaries, and generate a haiku one word at a time. The *count_syllables.py* program and *missing_words.json* file from **Chapter 8** will ensure *markov_haiku.py* uses the correct number of syllables for each line. You can download all these files from <https://www.nostarch.com/impracticalpython/> (**Chapter 9** folder). Be sure to keep them together in the same directory.

Setting Up

Listing 9-1 imports the necessary modules, then loads and prepares external files.

markov_haiku.py, part 1

```
❶ import sys
    import logging
    import random
    from collections import defaultdict
    from count_syllables import count_syllables

❷ logging.disable(logging.CRITICAL) # comment out to enable debugging messages
    logging.basicConfig(level=logging.DEBUG, format='%(message)s')

❸ def load_training_file(file):
```

```

        """Return text file as a string."""
        with open(file) as f:
            ❷ raw_haiku = f.read()
            return raw_haiku

    ❸ def prep_training(raw_haiku):
        """Load string, remove newline, split words on spaces, and return list."""
        corpus = raw_haiku.replace('\n', ' ').split()
        return corpus

```

Listing 9-1: Imports, loads, and prepares the training corpus

Start with the imports listed on separate lines ❶. You'll need `logging` in order to receive debugging messages, and `defaultdict` will help you build a dictionary from a list by creating a new key automatically, rather than throwing an error. You also import the `count_syllables` function from the `count_syllables.py` program you wrote in **Chapter 8**. You should be familiar with the rest of these imports.

Put the statement to disable `logging` right after the imports so you can find it easily. To see logging messages, you need to comment out this statement ❷. The following statement configures what you will see, as described in the previous section. I chose to omit the level designation from the display.

Next, define a function to load the training-corpus text file ❸. Use the built-in `read()` function to read the data as a string that the program can prepare before converting it to a list ❹. Return this string for use in the next function.

The `prep_training()` function ❺ takes the output from the `load_training_file()` function as an argument. It then replaces newline characters with spaces and splits the words into list items based on spaces. Finally, the function returns the corpus as a list.

Building Markov Models

The Markov models are simply Python dictionaries that use a word or word pair as a key and the word that immediately follows them as a value. The statistical frequency of trailing words is captured by repetition of the trailing word in the list of values—similar to sets, dictionaries can't have duplicate *keys*, but they can have duplicate *values*.

Listing 9-2 defines two functions. Both functions take the corpus as an argument and return a Markov model.

markov_haiku.py, part 2

```
❶ def map_word_to_word(corpus):
    """Load list & use dictionary to map word to word that follows."""
    ❷ limit = len(corpus) - 1
    ❸ dict1_to_1 = defaultdict(list)
    ❹ for index, word in enumerate(corpus):
        if index < limit:
            ❺ suffix = corpus[index + 1]
            dict1_to_1[word].append(suffix)
    ❻ logging.debug("map_word_to_word results for \"sake\" = %s\n",
                    dict1_to_1['sake'])
    ❼ return dict1_to_1

❽ def map_2_words_to_word(corpus):
    """Load list & use dictionary to map word-pair to trailing word."""
    ❾ limit = len(corpus) - 2
    dict2_to_1 = defaultdict(list)
    for index, word in enumerate(corpus):
        if index < limit:
            ❿ key = word + ' ' + corpus[index + 1]
            suffix = corpus[index + 2]
            dict2_to_1[key].append(suffix)
    logging.debug("map_2_words_to_word results for \"sake jug\" = %s\n",
                    dict2_to_1['sake jug'])
    return dict2_to_1
```

Listing 9-2: Defines functions that build Markov models of order 1 and 2

First, define a function to map every single word to its trailing word ❶. The program will use this function only to select the haiku's second word from the seed word. Its only parameter is the corpus list that the `prep_training()` function returns.

Set a limit so you can't pick the last word in the corpus ❷, because doing so would result in an index error. Now initialize a dictionary using `defaultdict` ❸. You want the dictionary values to be lists that hold all the suffixes you find, so use `list` as the argument.

Start looping through every word in the corpus, using `enumerate` to turn each word's index into an object ❹. Use a conditional and the `limit` variable to prevent choosing the last word as a key. Assign a variable named `suffix` that will represent the trailing word ❺. The value will be the index location of the current word plus 1—the next word in the list. Append this variable to the dictionary as a value of the current word.

To check that everything is working as planned, use `logging` to show the results *for a single key* ❻. There are thousands of words in the corpus, so you're not going to want to print them all. Choose a word that you know is in the corpus, like *sake*. Note that you are using the old string formatting with `%`, as it fits the current design of the logger. Finish by returning the dictionary ❼.

The next function, `map_2_words_to_word()`, is basically the same function, except it uses two consecutive words as the key and maps to trailing single words ❽. Important changes are to set the limit two words back from the end of the corpus ❾, make the key consist of two words with a space between them ❿, and add 2 to the index for the `suffix`.

Choosing a Random Word

The program won't be able to utilize a Markov model without a key, so either the user or the program must supply the first word in a simulated haiku. **Listing 9-3** defines a function that picks a first word at random, facilitating automated seeding.

```

❶ def random_word(corpus):
    """Return random word and syllable count from training corpus."""
    ❷ word = random.choice(corpus)
    ❸ num_syls = count_syllables(word)
    ❹ if num_syls > 4:
        random_word(corpus)
    else:
        ❺ logging.debug("random word & syllables = %s %s\n", word, num_syls)
        return (word, num_syls)

```

Listing 9-3: Randomly chooses a seed word to initiate the haiku

Define the function and pass it the `corpus` list ❶. Then assign a `word` variable and use the `random.choice()` method to pick a word from the corpus ❷.

Use the `count_syllables()` function from the `count_syllables` module to count the syllables in the word; store the count in the `num_syls` variable ❸. I'm not a fan of single-word lines in haiku, so don't allow the function to choose a word with more than four syllables (recall that the shortest haiku lines have five syllables). If this occurs, call the `random_word()` function recursively until you get an acceptable word ❹. Note that Python has a default maximum recursion depth of 1,000, but as long as you're using a proper haiku-training corpus, there's little chance you'll exceed that prior to finding a suitable word. If that were not the case, you could address this condition later by calling the function using a `while` loop.

If the word has fewer than five syllables, use `logging` to display the word and its syllable count ❺; then return the word and syllable count as a tuple.

Applying the Markov Models

To choose the single word that follows the seed word, use the Markov model of order 1. After that, the program should select all subsequent words using the order 2 model, which uses word pairs as keys. **Listing 9-4** defines a separate function for each of these actions.

```

❶ def word_after_single(prefix, suffix_map_1, current_sylys, target_sylys):
    """Return all acceptable words in a corpus that follow a single word."""
    ❷ accepted_words = []
    ❸ suffixes = suffix_map_1.get(prefix)
    ❹ if suffixes != None:
        ❺ for candidate in suffixes:
            num_sylys = count_syllables(candidate)
            if current_sylys + num_sylys <= target_sylys:
                ❻ accepted_words.append(candidate)
    ❼ logging.debug("accepted words after \"%s\" = %s\n",
                    prefix, set(accepted_words))
    return accepted_words

❽ def word_after_double(prefix, suffix_map_2, current_sylys, target_sylys):
    """Return all acceptable words in a corpus that follow a word pair."""
    accepted_words = []
    ❾ suffixes = suffix_map_2.get(prefix)
    if suffixes != None:
        for candidate in suffixes:
            num_sylys = count_syllables(candidate)
            if current_sylys + num_sylys <= target_sylys:
                accepted_words.append(candidate)
    logging.debug("accepted words after \"%s\" = %s\n",
                    prefix, set(accepted_words))
    ❿ return accepted_words

```

Listing 9-4: Two functions that select a word given a prefix, Markov model, and syllable count

Define a function named `word_after_single()` to select the next word in the haiku based on the preceding single seed word. For arguments, this function takes the preceding word, the Markov order 1 model, the current syllable count, and the target syllable count ❶.

Start an empty list to hold the acceptable words, which are the words that both follow the prefix and whose syllable count doesn't exceed the syllable target ❷. Call these trailing words `suffixes` and use the dictionary `get()`

method, which returns a dictionary value given a key, to assign them to the variable ❸. Rather than raising a `KeyError`, the `get()` method will return `None` if you request a key that isn't in the dictionary.

There is the extremely rare chance that the prefix will be the last word in a corpus and that it will be unique. In that case, there will be no suffixes. Use an `if` statement to anticipate this ❹. If there are no suffixes, the function that calls `word_after_single()`, which you'll define in the next section, chooses a new prefix.

Each of the suffixes represents a *candidate* word for the haiku, but the program hasn't yet determined whether a candidate will "fit." So use a `for` loop, the `count_syllables` module, and an `if` statement to determine whether adding the word to the line violates the target number of syllables per line ❺. If the target isn't exceeded, append the word to the list of accepted words ❻. Display the acceptable words in a `logging` message and then return it ❼.

The next function, `word_after_double()`, is like the previous function except that you pass it word pairs and the Markov order 2 model (`suffix_map_2`) ❽ and get the suffixes from this dictionary ❾. But just like the `word_after_single()` function, `word_after_double()` returns a list of acceptable words ❿.

Generating the Haiku Lines

With all the helper functions ready, you can define the function that actually writes the lines of the haiku. The function can build either the whole haiku or just update lines two or three. There are two paths to take: one to use when the program has at most a one-word suffix to work with and another for every other situation.

Building the First Line

Listing 9-5 defines the function that writes haiku lines and initiates the haiku's first line.

```

❶ def haiku_line(suffix_map_1, suffix_map_2, corpus, end_prev_line, target_syls
    """Build a haiku line from a training corpus and return it."""
❷ line = '2/3'
    line_syls = 0
    current_line = []
❸ if len(end_prev_line) == 0: # build first line
    ❹ line = '1'
    ❺ word, num_syls = random_word(corpus)
        current_line.append(word)
        line_syls += num_syls
    ❻ word_choices = word_after_single(word, suffix_map_1,
                                        line_syls, target_syls)

    ❼ while len(word_choices) == 0:
        prefix = random.choice(corpus)
        logging.debug("new random prefix = %s", prefix)
        word_choices = word_after_single(prefix, suffix_map_1,
                                        line_syls, target_syls)

    ❽ word = random.choice(word_choices)
        num_syls = count_syllables(word)
        logging.debug("word & syllables = %s %s", word, num_syls)
❹ line_syls += num_syls
        current_line.append(word)
❺ if line_syls == target_syls:
        end_prev_line.extend(current_line[-2:])
        return current_line, end_prev_line

```

Listing 9-5: Defines the function that writes haiku lines and initiates the first line

Define a function that takes as arguments both Markov models, the training corpus, the last word pair from the end of the preceding line, and the target number of syllables for the current line ❶. Immediately use a variable to specify which haiku lines are being simulated ❷. Most of the processing will be for lines two and three (and possibly the last part of line one), where you will be working with an existing word-pair prefix, so let these represent the base case. After this, start a counter for the running

total of syllables in the line and start an empty list to hold the words in the current line.

Use an `if` statement that's `True` under the condition that the `end_prev_line` parameter's length—the number of syllables in the previous line's last two words—is 0, meaning there was no preceding line and you are on line one ❸. The first statement in that `if` block changes the `line` variable to 1 ❹.

Choose the initial seed word and get its syllable count by calling the `random_word()` function ❺. By assigning the `word` and `num_sylls` variables together, you are “unpacking” the `(word, num_sylls)` tuple that the `random_word()` function returns. Functions end at `return` statements, so returning tuples is a great way to return multiple variables. In a more advanced version of this program, you could use generator functions with the `yield` keyword, as `yield` returns a value without relinquishing control of execution.

Next, append the `word` to `current_line` and add the `num_sylls` to the running total. Now that you have a seed, collect all the seed's possible suffixes with the `word_after_single()` function ❻.

If there are no acceptable words, start a `while` loop to handle this situation. This loop will continue until a non-empty list of acceptable word choices has been returned ❼. The program will choose a new prefix—a ghost prefix—using the `random` module's `choice` method. (Remember that this prefix will not become part of the haiku but is used only to reaccess the Markov model.) Inside the `while` loop, a logging message will let you know which ghost prefix has been chosen. Then the program will call `word_after_single()` function once more.

Once the list of acceptable words is built, use `choice` again to select a word from the `word_choices` list ❽. Because the list may include duplicate words, this is where you see the statistical impact of the Markov model. Follow by counting the syllables in the word and logging the results.

Add the syllable count to the line's running total and append the word to the `current_line` list ❾.

If the number of syllables in the first two words equals 5 ❶, name a variable `end_prev_line` and assign it the last two words of the previous line; this variable is the prefix for line two. Finally, return the whole line and the `end_prev_line` variable.

If the target number of syllables for the first line hasn't been reached, the program will jump to the `while` loop in the next section to complete the line.

Building the Remaining Lines

In **Listing 9-6**, the last part of the `haiku_line()` function addresses the case where the haiku already contains a word-pair prefix that the program can use in the Markov order 2 model. The program uses it to complete line one—assuming the first two words didn't already total five syllables—and to build lines two and three. The user will also be able to regenerate lines two or three, after a complete haiku has been written.

markov_haiku.py, part 6

```
❶ else: # build lines 2 and 3
    ❷ current_line.extend(end_prev_line)

    ❸ while True:
        logging.debug("line = %s\n", line)
        ❹ prefix = current_line[-2] + ' ' + current_line[-1]
        ❺ word_choices = word_after_double(prefix, suffix_map_2,
                                           line_sylys, target_sylys)

        ❻ while len(word_choices) == 0:
            index = random.randint(0, len(corpus) - 2)
            prefix = corpus[index] + ' ' + corpus[index + 1]
            logging.debug("new random prefix = %s", prefix)
            word_choices = word_after_double(prefix, suffix_map_2,
                                             line_sylys, target_sylys)

        word = random.choice(word_choices)
        num_sylys = count_syllables(word)
        logging.debug("word & syllables = %s %s", word, num_sylys)
```

```

    ⑦ if line_syls + num_syls > target_syls:
        continue
    elif line_syls + num_syls < target_syls:
        current_line.append(word)
        line_syls += num_syls
    elif line_syls + num_syls == target_syls:
        current_line.append(word)
        break

    ⑧ end_prev_line = []
    end_prev_line.extend(current_line[-2:])

    ⑨ if line == '1':
        final_line = current_line[:]
    else:
        final_line = current_line[2:]

    return final_line, end_prev_line

```

Listing 9-6: Uses a Markov order 2 model to complete the function that writes haiku lines

Start with an `else` statement that's executed if there is a suffix ❶. Since the last part of the `haiku_line()` function has to handle line one as well as lines two and three, use a trick where you add the `end_prev_line` list (built outside the conditional at step ❸) to the `current_line` list ❷. Later, when you add the finalized line to the haiku, you'll discard this leading word pair.

Start a `while` loop that continues until the target syllable count for the line has been reached ❸. The start of each iteration begins with a debugging message that informs you of the path the loop is evaluating: `'1'` or `'2/3'`.

With the last two words of the previous line added to the start of the current line, the last two words of the current line will always be the prefix ❹.

Using the Markov order 2 model, create a list of acceptable words ❺. In the event the list is empty, the program uses the ghost prefix process ❻.

Evaluate what to do next using the syllable count ⑦. If there are too many syllables, use a `continue` statement to restart the `while` loop. If there aren't enough syllables, append the word and add its syllable count to the line's syllable count. Otherwise, append the word and end the loop.

Assign the last two words in the line to the `end_prev_line` variable so the program can use it as a prefix for the next line ⑧. If the current path is line '1', copy the current line to a variable called `final_line`; if the path is line '2/3', use index slicing to exclude the first two words before assigning to `final_line` ⑨. This is how you remove the initial `end_prev_line` word pair from lines two or three.

Writing the User Interface

Listing 9-7 defines the *markov_haiku.py* program's `main()` function, which runs the setup functions and the user interface. The interface presents the user with a menu of choices and displays the resulting haiku.

markov_haiku.py, part 7

```
def main():
    """Give user choice of building a haiku or modifying an existing haiku."""
    intro = """\n
    A thousand monkeys at a thousand typewriters...
    or one computer...can sometimes produce a haiku.\n"""
    print("{}".format(intro))

    ❶ raw_haiku = load_training_file("train.txt")
    corpus = prep_training(raw_haiku)
    suffix_map_1 = map_word_to_word(corpus)
    suffix_map_2 = map_2_words_to_word(corpus)
    final = []

    choice = None
    ❷ while choice != "0":

        ❸ print(
            """
```

Japanese Haiku Generator

```
0 - Quit
1 - Generate a Haiku
2 - Regenerate Line 2
3 - Regenerate Line 3
"""
)

④ choice = input("Choice: ")
print()

# exit
⑤ if choice == "0":
    print("Sayonara.")
    sys.exit()

# generate a full haiku
⑥ elif choice == "1":
    final = []
    end_prev_line = []
    first_line, end_prev_line1 = haiku_line(suffix_map_1, suffix_map_2,
                                             corpus, end_prev_line, 5)

    final.append(first_line)
    line, end_prev_line2 = haiku_line(suffix_map_1, suffix_map_2,
                                       corpus, end_prev_line1, 7)

    final.append(line)
    line, end_prev_line3 = haiku_line(suffix_map_1, suffix_map_2,
                                       corpus, end_prev_line2, 5)

    final.append(line)

# regenerate line 2
⑦ elif choice == "2":
    if not final:
        print("Please generate a full haiku first (Option 1).")
        continue
    else:
        line, end_prev_line2 = haiku_line(suffix_map_1, suffix_map_2,
```

```

corpus, end_prev_line1, 7)

    final[1] = line

    # regenerate line 3
    ❸ elif choice == "3":
        if not final:
            print("Please generate a full haiku first (Option 1).")
            continue
        else:
            line, end_prev_line3 = haiku_line(suffix_map_1, suffix_map_2,
                                              corpus, end_prev_line2, 5)

            final[2] = line

    # some unknown choice
    ❹ else:
        print("\nSorry, but that isn't a valid choice.", file=sys.stderr)
        continue

    ❺ # display results
    print()
    print("First line = ", end="")
    print(' '.join(final[0]), file=sys.stderr)
    print("Second line = ", end="")
    print(" ".join(final[1]), file=sys.stderr)
    print("Third line = ", end="")
    print(" ".join(final[2]), file=sys.stderr)
    print()

    input("\n\nPress the Enter key to exit.")

if __name__ == '__main__':
    main()

```

Listing 9-7: Starts the program and presents a user interface

After the introduction message, load and prep the training corpus and build the two Markov models. Then create an empty list to hold the final

haiku ❶. Next, name a `choice` variable and set it to `None`. Start a `while` loop that continues until the user chooses choice 0 ❷. By entering 0, the user has decided to quit the program.

Use a `print()` statement with triple quotes to display a menu ❸, and then get the user's choice ❹. If the user chooses 0, exit and say goodbye ❺. If the user chooses 1, they want the program to generate a new haiku, so reinitialize the `final` list and the `end_prev_line` variable ❻. Then call the `haiku_line()` function for all three lines and pass it the correct arguments—including the target syllable count for each line. Note that the `end_prev_line` variable name changes with each line; for example, `end_prev_line2` holds the final two words of line two. The last variable, `end_prev_line3`, is just a placeholder so you can reuse the function; in other words, it is never put to use. Each time the `haiku_line()` function is called, it returns a line that you need to append to the `final` list.

If the user chooses 2, the program regenerates the second line ❼. There needs to be a full haiku before the program can rebuild a line, so use an `if` statement to handle the user jumping the gun. Then call the `haiku_line()` function, making sure to pass it the `end_prev_line1` variable to link it to the preceding line, and set the syllable target to seven syllables. Insert the rebuilt line in the `final` list at index 1.

Repeat the process if the user chooses 3, only make the syllable target 5 and pass `end_prev_line2` to the `haiku_line()` function ❽. Insert the line at index 2 in `final`.

If the user enters anything not on the menu, let them know and then continue the loop ❾. Finish by displaying the haiku. Use the `join()` method and `file=sys.stderr` for an attractive printout in the shell ❿.

End the program with the standard code for running the program as a module or in stand-alone mode.

The Results

To evaluate a poetry-writing program, you need a way to measure something subjective—whether or not poems are “good”—using objective cri-

teria. For the *markov_haiku.py* program, I propose the following categories based on two criteria, originality and humanness:

Duplicate Verbatim duplication of a haiku in the training corpus.

Good A haiku that is—at least to some people—indistinguishable from a haiku written by a human poet. It should represent the initial result or the result of regenerating line two or three a few times.

Seed A haiku that has merit but that many would suspect was written by a computer, or a haiku that you could transform into a good haiku by changing or repositioning no more than two words (described in more detail later). It may require multiple regenerations of lines two or three.

Rubbish A haiku that is clearly a random amalgam of words and has no merit as a poem.

If you use the program to generate a large number of haiku and place the results in these categories, you'll probably end up with the distribution in **Figure 9-3**. About 5 percent of the time, you'll duplicate an existing haiku in the training corpus; 10 percent of the time, you'll produce a good haiku; around 25 percent will be passable or fixable haiku; and the rest will be rubbish.

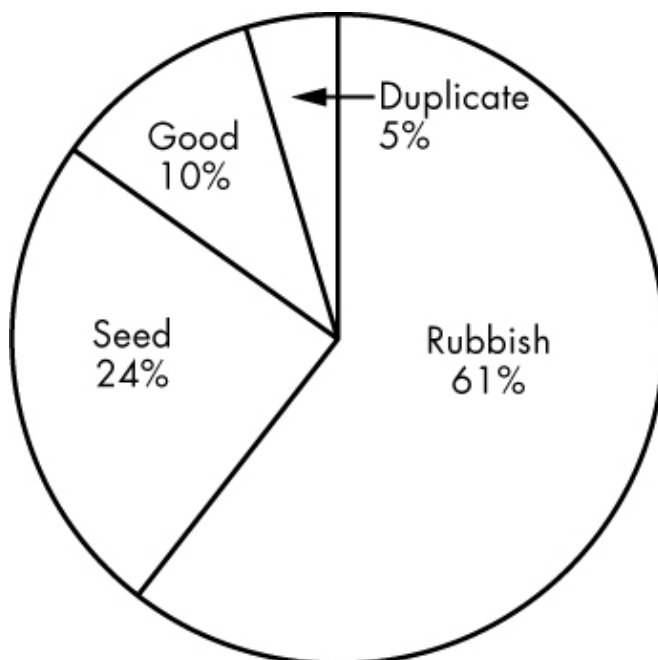


Figure 9-3: Subjective results of generating 500 haiku using markov_haiku.py

Considering how simple the Markov process is, the results in **Figure 9-3** are impressive. To quote Charles Hartman once again, “Here is language creating itself out of nothing, out of mere statistical noise. . . . We can watch sense evolve and meaning stagger up onto its own miraculous feet.”

Good Haiku

The following are some examples of simulated haiku classified as “good.” In the first example, the program has subtly—you might say “skillfully,” if you didn’t know an algorithm was responsible—altered my haiku from **Chapter 8** to produce a new haiku with the same meaning.

Cloudbanks that I let

Myself pretend are distant

Mountains faraway

In the next example, the program has managed to duplicate a common theme in traditional haiku: the juxtaposition of images or ideas.

The mirror I stare

Into shows my father’s face

An old silent pond

In this case, you find out that the mirror is really the surface of a still pond, though you may interpret the face itself as the pond.

Running the program is a little like panning for gold: sometimes you find a nugget. The haiku on the left was written by Ringai over 300 years ago. In the haiku on the right, the program has made subtle changes so that the poem now evokes images of a late spring freeze—a setback for the progress of the season.

In these dark waters

Drawn up from my frozen well

Waters drawn up from

My frozen well glittering

Glittering of Spring
—*Ringai*

Of Spring standing still
—*Python*

The following are examples of a few more “good” haiku. The first one is remarkable since it was built from three separate haiku in the training corpus, yet maintains a clear contextual thread throughout.

As I walk the path
Eleven brave knights canter
Through the stormy woods

A line flip-flapping
Across the dark crimson sky
On this winter pond

Such a thing alive
Rusted gate screeches open
Even things feel pain

The stone bridge! Sitting
Quietly doing nothing
Yet Spring comes grass grows

Dark sky oh! Autumn
Snowflakes! A rotting pumpkin
Collapsed and covered

Desolate moors fray
Black cloudbank, broken, scatters
In the pines, the graves

Seed Haiku

The concept of computers helping humans write poetry has been around for some time. Poets often “prime the pump” by imitating earlier poems, and there’s no reason a computer can’t provide first drafts as part of a cyber-partnership. Even a fairly poor computer creation has the potential to “seed” the creative process and help a human overcome writer’s block.

The following are three examples of seed haiku from the *markov_haiku.py* program. On the left is the not-quite-right computer-generated haiku. On the right is the version that I adjusted. I changed only a single word, highlighted in bold, in each.

My life must end like
Another flower what a
Hungry wind **it** is

My life must end like
Another flower what a
Hungry wind is **death**

The dock floating in
The hot caressing night just
Before the dawn **old**

The dock floating in
The hot caressing night just
Before the dawn **rain**

Moonrise on the grave
And my old sadness a sharp
Shovel thrust **the** stars

Moonrise on the grave
And my old sadness a sharp
Shovel thrust **of** stars

The last has cryptic meaning but seems to work, as it is filled with natural associations (moon and stars, grave and shovel, grave and sadness). At any rate, you shouldn't fret too much over meaning. To paraphrase T.S. Eliot: meaning is like the meat the burglar throws the dog, to keep the mind diverted while the poem does its work!

Summary

It took two chapters, but you now have a program that can simulate Japanese haiku written by the masters—or at least provide a useful starting point for a human poet. In addition, you applied the `logging` module to monitor what the program was doing at key steps.

Further Reading

Virtual Muse: Experiments in Computer Poetry (Wesleyan University Press, 1996) by Charles O. Hartman is an engaging look at the early collabora-

tion between humans and computers to write poetry.

If you want to know more about Claude Shannon, check out *A Mind at Play: How Claude Shannon Invented the Information Age* (Simon & Schuster, 2017) by Jimmy Soni and Rod Goodman.

You can find a digital version of *Japanese Haiku: Two Hundred Twenty Examples of Seventeen-Syllable Poems* (The Peter Pauper Press, 1955), translated by Peter Beilenson, online at Global Grey (<https://www.globalgreybooks.com/>).

In the paper “Gaiku: Generating Haiku with Word Association Norms” (Association for Computational Linguistics, 2009), Yael Netzer and coauthors explore the use of word association norms (WANs) to generate haiku. You can build WAN corpora by submitting trigger words to people and recording their immediate responses (for example, *house* to *fly*, *arrest*, *keeper*, and so on). This results in the kind of tightly linked, intuitive relationships characteristic of human-generated haiku. You can find the paper online at

<http://www.cs.brandeis.edu/~marc/misc/proceedings/naacl-hlt-2009/CALC-09/pdf/CALC-0905.pdf>.

Automate the Boring Stuff with Python (No Starch Press, 2015) by Al Sweigart has a useful overview chapter on debugging techniques, including logging.

Challenge Projects

I’ve described some suggestions for spin-off projects in this section. As with all challenge projects, you’re on your own—no solutions are provided.

New Word Generator

In his award-winning 1961 sci-fi novel, *Stranger in a Strange Land*, author Robert A. Heinlein invented the word *grok* to represent deep, intuitive understanding. This word moved into the popular culture—especially the culture of computer programming—and is now in the *Oxford English Dictionary*.

Coming up with a new word that sounds legitimate is not easy, in part because humans are so anchored to the words we already know. But computers don't suffer from this affliction. In *Virtual Muse*, Charles Hartman observed that his poetry-writing program would sometimes create intriguing letter combinations, such as *runkin* or *avatheformitor*, that could easily represent new words.

Write a program that recombines letters using Markov order 2, 3, and 4 models and use the program to generate interesting new words. Give them a definition and start applying them. Who knows—you may coin the next *frickin*, *frabjous*, *chortle*, or *trill*!

Turing Test

According to Alan Turing, “A computer would deserve to be called intelligent if it could deceive a human into believing that it was human.” Use your friends to test haiku generated by the *markov_haiku.py* program. Mix the computer haiku with a few haiku written by the masters or yourself. Since computer haiku are often enjambed, be careful to choose human haiku that are also enjambed in order to deny your cleverer friends a free ride. Using lowercase letters and minimal punctuation for all the haiku also helps. I've provided an example, using Facebook, in **Figure 9-4**.

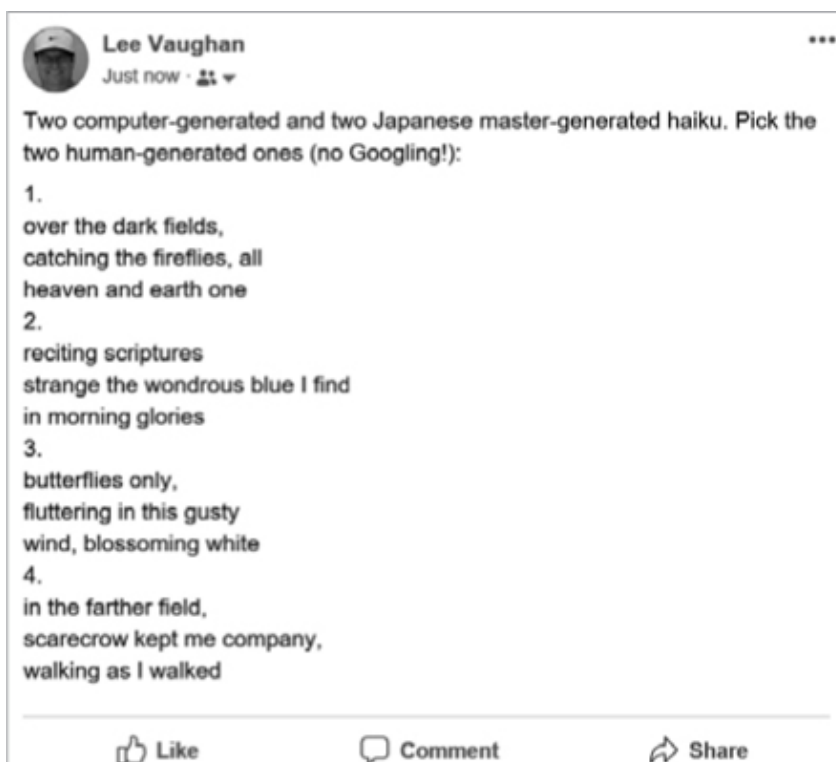


Figure 9-4: Example Turing test experiment post on Facebook

Unbelievable! This Is Unbelievable! Unbelievable!

President Trump is famous for speaking in short, simple sentences that use “the best words,” and short, simple sentences are great for haiku. In fact, the *Washington Post* published some inadvertent haiku found in some of his campaign speeches. Among these were:

He’s a great, great guy.
I saw him the other day.
On television.

They want to go out.
They want to lead a good life.
They want to work hard.

We have to do it.
And we need the right people.
So Ford will come back.

Use online transcripts of Donald Trump’s speeches to build a new training corpus for the *markov_haiku.py* program. Remember that you’ll need to revisit **Chapter 8** and build a new “missing words” dictionary for any words not in the *Carnegie Mellon University Pronouncing Dictionary*. Then rerun the program and generate haiku that capture this moment in history. Save the best ones and revisit the Turing test challenge to see if your friends can separate your haiku from true Trump quotes.

To Haiku, or Not to Haiku

William Shakespeare wrote many famous phrases that fit the syllabic structure of haiku, such as “all our yesterdays,” “dagger of the mind,” and “parting is such sweet sorrow.” Use one or more of the Bard’s plays as a training corpus for the *markov_haiku.py* program. The big challenge here will be counting the syllables for all that olde English.

Markov Music

If you’re musically inclined, perform an online search for “composing music with Markov chains.” You should find a wealth of material on using

Markov chain analysis to compose music, using the notes from existing songs as a training corpus. The resulting “Markov music” is used like our seed haiku—as inspiration for human songwriters.